

Master Thesis Technical Report: Phase I — Dataset Curation and Pipeline Validation

Ali Bakhshayesh

May 3, 2026

Abstract

This report consolidates thesis work completed in Phase I (Initial Cohort), focused on building and validating the extraction pipeline for developer-reputation candidate metrics under defined evidence and reproducibility constraints. The document reports repository and version-window curation, pipeline outputs, and post-extraction data cleaning outcomes. The comparative evaluation phase (Phase II) is intentionally deferred, and no proof-of-concept (PoC) metric is selected at this stage. PoC metric selection is explicitly performed after Phase II comparative evaluation using the feasibility rubric.

Contents

1	Abbreviations and Notation	6
2	Introduction, Scope, and Research Roadmap	6
3	Candidate Metrics (Shortlist)	7
4	Methodology: Comparison Criteria	7
4.1	Evaluation unit and normalization policy	7
4.2	C1 Stability	8
4.3	C2 Interpretability	8
4.4	C3 Resistance to gaming	8
4.5	C4 Long-term suitability	8
4.6	Criterion-to-output mapping	9
4.7	Decision protocol (predefined, not executed in Phase I)	9
5	DESTINO Alignment and Evidence Boundary	9
5.1	Implications for the Current Report	10
6	Extended Feasibility Rubric (Detailed)	10
6.1	Smart-Contract Feasibility Rubric	10
7	Future DESTINO PoC Integration Blueprint	13
7.1	Design Objective and Boundary	14
7.2	Metric-Agnostic Record Shape	14
7.3	Transaction and Replay Flow	14
7.4	How Phase I Outputs Support the Blueprint	15
8	Phase I: Dataset Freeze and Version-Window Definition	15
8.1	Repository Freeze	15
8.2	Tag Freeze and Consecutive Windows	15
9	Phase I: Metric Extraction Pipeline	16
9.1	Primary CSV outputs	16
9.2	Script-to-output mapping	17
9.3	Metric definitions (implementation-level)	17
9.4	Error handling and traceability	18
10	Phase I: Data Integrity Verification and Cleaning	18
10.1	Sanity checks	18
10.2	Canonical identity	18
10.3	Bot labeling	18
10.4	Known limitation: commit-date/tag-date drift	18
10.5	Issue-resolution ledger	19
11	Initial Cohort Extraction Summary (Phase I Deliverables)	19
12	Detailed Descriptive Results for M1–M4 (Initial Cohort)	20
12.1	M1 Snapshot size evolution (LOC/SLOC)	20
12.2	M2 Churn and M3 cadence intensity by project	20
12.3	Bot-sensitivity outputs	21
12.4	Duration-normalized analysis outputs	21
12.5	M3 Developer activity-day profile	21

12.6 M4 Ownership and concentration profile	22
13 Phase I Execution Ledger (Detailed, Step-Mapped)	22
13.1 Cleaning and validation outcomes	23
14 Reproducibility Protocol and Data Lineage	24
14.1 Artifact layout	24
14.2 Execution protocol	24
14.3 Lineage checkpoints	24
14.4 Automated validation evidence	25
15 Threats to Validity	25
15.1 Construct validity	25
15.2 Internal validity	25
15.3 External validity	25
15.4 Conclusion validity	25
16 Phase I Conclusion	26
17 Planned Next Phase: Comparative Metric Evaluation	26
17.1 Phase II execution plan	26
17.2 Decision gate criteria	26

List of Tables

1	Abbreviations and working notation used throughout the report.	6
2	Candidate shortlist. All metrics are computed and compared before selecting a single PoC metric.	7
3	Operational mapping from methodology criteria to evidence sources.	9
4	Alignment between DESTINO evidence requirements and current Phase I artifacts. . . .	10
5	Feasibility dimensions with detailed scoring rules (D1–D5).	12
6	Feasibility dimensions with detailed scoring rules (D6–D10).	12
7	Feasibility rubric status table. All scores are deferred to Phase II comparative evaluation.	13
8	Metric-agnostic record shape for the future DESTINO PoC.	14
9	Frozen Initial Cohort dataset (<code>dataset_freeze_round1.csv</code>).	15
10	Initial Cohort version coverage extracted from <code>versions.csv</code> and <code>windows.csv</code>	16
11	Window-duration statistics.	16
12	Executable mapping from pipeline scripts to canonical Initial Cohort outputs.	17
13	Initial Cohort issue ledger and implemented cleaning/validation actions.	19
14	Initial Cohort summary from cleaned datasets (commits/churn totals per project; bot rows flagged separately).	19
15	Initial Cohort global extraction totals from all-account and human-only clean outputs. . .	19
16	Snapshot size statistics (code lines) across the 8 frozen tags per project (M1).	20
17	M1 evolution across frozen tags (values from <code>out/loc_per_tag.csv</code> ; first/last are chronological tag endpoints).	20
18	M2/M3 human-only project-level intensity.	20
19	Bot impact summary from <code>out/bot_summary_by_project.csv</code> . Percentages are relative to all-account project totals.	21
20	Short-window sensitivity flags. Windows below seven days are retained but marked as <code>sensitivity_only</code>	21
21	Highest churn/day windows from <code>out/metrics_window_totals_normalized.csv</code> . Very short windows can inflate rates and are explicitly flagged for sensitivity analysis. . .	21
22	M3 human-only activity profile.	21
23	M4 human-only share concentration means from <code>out/ownership_summary_human.csv</code>	22
24	M4 human-only inequality indicators from <code>out/ownership_summary_human.csv</code>	22
25	Detailed execution ledger for Phase I Initial Cohort pipeline operations.	23
26	Initial Cohort data lineage checkpoints used for reproducibility and auditing.	24

List of Figures

1 Abbreviations and Notation

Term	Meaning in this report
PoC	Proof-of-concept smart-contract implementation (selected after Phase II only).
DESTINO	Target evidence-based developer-reputation framework used to define the smart-contract evidence boundary.
LOC/SLOC	Total/source line counts used as snapshot-size indicators (M1).
Churn	Added plus deleted lines in a window (M2).
Cadence	Activity continuity over time (commits, active days, and gaps) (M3).
Ownership concentration	Distribution of developer contribution shares (M4).
Window	Consecutive tag interval ($T_k \rightarrow T_{k+1}$) in chronological order.
All-account	Metric population including both human and automated Git author identities.
Human-only	Metric population excluding rows flagged as automated accounts via <code>is_bot</code> .
Initial Cohort	Current Phase I dataset batch with 4 repositories, 8 tags each, and 7 windows each.

Table 1: Abbreviations and working notation used throughout the report.

2 Introduction, Scope, and Research Roadmap

This thesis studies software-quality indicators as evidence for developer reputation systems. The objective of the current phase is methodological: construct a reproducible multi-project dataset, extract candidate metrics with transparent rules, and produce analysis-ready outputs for comparative evaluation. No proof-of-concept (PoC) metric is selected in this phase; metric selection is deferred to Phase II.

Research Questions

- **RQ1 (Stability):** Which candidate indicators are stable across versions and comparable across projects?
- **RQ2 (Interpretability and gaming):** Which indicators are understandable and robust against manipulation?
- **RQ3 (Smart-contract feasibility):** Which indicator is the strongest PoC candidate under implementation constraints?

Document Structure

The report is organized as follows:

- Sections 3–7 define candidate metrics, comparison criteria, DESTINO alignment, feasibility scoring, and the future PoC integration boundary.
- Sections 8–11 document dataset curation, extraction, cleaning, and descriptive outputs for the Initial Cohort.
- Sections 14–16 provide reproducibility and validity considerations.
- Section 17 defines the comparative evaluation plan and decision gate for PoC metric selection.

Contributions

- A shortlist of candidate metrics and operational comparison criteria.
- A fully reproducible Initial-Cohort dataset (frozen repositories, tags, and consecutive windows) and extraction pipeline.
- Cleaned, validated metric datasets (developer-window and window totals) and ownership/concentration statistics.
- A detailed smart-contract feasibility rubric to support later PoC metric selection.
- A DESTINO-aligned evidence model showing how Phase I outputs map to a future metric-agnostic PoC.

3 Candidate Metrics (Shortlist)

We compare a minimal set of candidate indicator families that are (i) computable from versioned artifacts and Git history, (ii) interpretable and comparable across heterogeneous projects, and (iii) compatible with hash-anchored evidence and compact on-chain storage. The shortlist is also informed by existing smart-contract metric and complexity literature [10, 8]. The comparative study evaluates all candidates; the PoC metric is selected only after evaluation.

Candidate	Usage in this thesis	Why chosen (one sentence)
M1 Snapshot size (LOC/SLOC)	Per-tag size baseline for normalization and context.	Simple, interpretable baseline; efficient to compute, store, and audit.
M2 Patch complexity proxy (churn)	Added/deleted lines per release window and developer.	Minimal proxy for change magnitude; aggregation-friendly and auditable.
M3 Activity cadence	Commits/activity-days per window; inactivity gaps.	Captures continuity over time without requiring platform-specific PR data.
M4 Ownership / concentration	Developer shares of churn/commits; top-1/top-3 and Gini concentration.	Separates core maintainers from occasional contributors; supports stability analysis.
M5 Maintenance responsiveness (design extension)	Documented schema/logic; not implemented in baseline PoC.	Included for completeness, but deferred to keep baseline PoC scope controlled.

Table 2: Candidate shortlist. All metrics are computed and compared before selecting a single PoC metric.

4 Methodology: Comparison Criteria

We compare candidates using four criteria: stability, interpretability, resistance to gaming, and long-term suitability. The comparative phase (Phase II) applies these criteria uniformly to M1–M5 before any PoC metric decision.

4.1 Evaluation unit and normalization policy

The primary analysis unit is (`project_id`, `window_id`, `developer`) for developer-level metrics and (`project_id`, `window_id`) for aggregate metrics. Because window durations and project sizes vary, the pipeline now produces both raw and normalized values:

- per-day normalization for cadence-sensitive measures;
- per-KLOC normalization where size effects can dominate;
- within-project shares (ownership) to control for absolute scale.

Windows shorter than seven days are retained for auditability but flagged as `sensitivity_only`; Phase II reports both full-dataset results and robust results excluding those very short windows when duration-sensitive metrics are evaluated.

Automation is handled as an explicit population rule. Rows flagged with `is_bot` remain in the clean all-account tables for auditability and project-activity totals. However, developer-behavior metrics use human-only outputs as the primary population: M3 cadence and M4 ownership/concentration are evaluated from `out/metrics_window_dev_human.csv` and `out/ownership_summary_human.csv`. Bot summaries are reported as sensitivity evidence rather than silently mixed into maintainer-behavior claims.

4.2 C1 Stability

Stability is evaluated across consecutive version windows. We measure volatility (e.g., dispersion across windows) and rank stability (e.g., correlation of developer rankings between adjacent windows). Cross-project comparison is performed after simple normalizations (e.g., per day or per KLOC) to reduce artifacts from project size and release cadence. Planned outputs include dispersion statistics, adjacent-window rank correlations, and sensitivity checks on timestamp semantics.

4.3 C2 Interpretability

Interpretability is assessed via semantic clarity (plain meaning), unit clarity (counts/shares/time), and normalization clarity. Metrics that require complex tooling or ambiguous definitions receive lower interpretability.

4.4 C3 Resistance to gaming

We analyze attack surfaces and manipulation cost per metric. Since single metrics can be misleading, we avoid strong causal claims and interpret gaming risks comparatively (e.g., commit-splitting inflates frequency but becomes detectable when churn remains low). The threat model follows common public-ledger constraints and manipulation tradeoffs discussed in blockchain and smart-contract studies [1, 10, 8].

4.5 C4 Long-term suitability

We assess whether each metric remains informative over long histories, across lifecycle phases, and whether it supports incremental updates without full recomputation.

4.6 Criterion-to-output mapping

Criterion	Planned Phase II evidence	Primary data source(s)
C1 Stability	Window-level volatility, adjacent-window rank correlation, robustness to normalization choices.	out/metrics_window_dev_normalized.csv, out/metrics_window_totals_human_normalized.csv, out/window_duration_flags.csv
C2 Inter-pretability	Structured rubric (semantic clarity, unit clarity, normalization clarity, tooling transparency).	Metric definitions plus extracted fields in out/ artifacts
C3 Gaming resistance	Attack-surface matrix and manipulation detectability checks under multi-metric reading.	out/metrics_window_dev_human.csv, out/ownership_dev_shares_human.csv, out/bot_summary_by_project.csv
C4 Long-term suitability	Informativeness over time, lifecycle robustness, and incremental update feasibility.	out/loc_per_tag.csv, data/windows.csv, feasibility rubric dimensions

Table 3: Operational mapping from methodology criteria to evidence sources.

4.7 Decision protocol (predefined, not executed in Phase I)

The decision protocol is fixed before Phase II analysis:

1. compute comparative evidence for all candidates under C1–C4;
2. score smart-contract feasibility using the rubric in Section 6.1;
3. eliminate any candidate that violates feasibility exclusion criteria;
4. select one PoC metric with the best overall tradeoff between empirical behavior and feasibility.

No step of this decision protocol is executed in Phase I.

5 DESTINO Alignment and Evidence Boundary

DESTINO is the reference deployment context for the thesis: software-quality evidence should support developer reputation claims through signed submissions, immutable records, and replayable verification. In this report, DESTINO is not treated as a reason to move all computation on-chain. Instead, it defines the evidence boundary that every candidate metric must satisfy before a proof-of-concept metric can be selected.

The working DESTINO-aligned evidence model has four parts:

- a developer-facing submission containing project/archive metadata, software-type tags, storage-persistence metadata, and a cryptographic commitment to the submitted artifact;
- an off-chain measurement pipeline that retrieves the committed artifact, verifies integrity, and computes the selected metric;
- compact on-chain records containing the submission reference, metric value, and evidence hash;
- public auditability through replay: retrieve the same artifact, rerun the declared script/configuration, and compare the result with the committed metric record.

This boundary keeps expensive repository traversal, Git history analysis, and tabular validation off-chain, while the smart-contract layer stores only bounded metadata, commitments, and metric outputs. This matches common blockchain engineering constraints: large artifacts and unbounded loops are poor on-chain candidates, while hash commitments and compact records are auditable and storage-bounded [1].

5.1 Implications for the Current Report

The Phase I report is therefore not only a data-extraction report. It prepares the exact evidence artifacts that a later DESTINO PoC would need. The alignment is summarized in Table 4.

DESTINO need	Phase I artifact or rule	Why it matters
Stable artifact identity	data/dataset_freeze_round1.csv, data/versions.csv, frozen commit hashes	Metric evidence can be tied to explicit projects, tags, and repository states.
Replayable measurement interval	data/windows.csv	Each metric value has a deterministic release-window boundary.
Compact metric evidence	out/loc_per_tag.csv, clean metric tables, normalized tables, human-only M3/M4 tables	The future contract can store selected values without storing raw repositories or diffs.
Audit and validation evidence	logs/*.log, cleaning checks, duration flags, bot-sensitivity summaries	A verifier can inspect how raw extraction became analysis-ready evidence.
Smart-contract selection discipline	Criteria C1–C4 plus the feasibility rubric	The selected PoC metric must be empirically useful and feasible under bounded contract execution.

Table 4: Alignment between DESTINO evidence requirements and current Phase I artifacts.

Two current design decisions are especially important for DESTINO alignment. First, duration-normalized outputs and `sensitivity_only` flags prevent release cadence artifacts from being silently recorded as comparable metric evidence. Second, human-only M3/M4 outputs prevent automated accounts from being interpreted as human maintainer behavior. The all-account tables remain useful as an audit trail, but the primary developer-behavior evidence is explicitly human-only.

No PoC metric is selected in Phase I. DESTINO alignment at this stage means the pipeline produces evidence that can later be committed, audited, and scored; it does not mean that any current metric has already passed the comparative and feasibility gate.

6 Extended Feasibility Rubric (Detailed)

6.1 Smart-Contract Feasibility Rubric

6.1.1 Why feasibility must be a first-class criterion

This thesis evaluates multiple candidate metrics for evidence-based developer reputation. However, the project explicitly requires that feasibility as a smart contract is considered as a selection criterion and that a proof-of-concept (PoC) is implemented for one chosen metric. The project scope further constrains the design by prescribing an evidence workflow where developers submit a signed form containing a link and a hash of an archive; the smart contract verifies the submission, retrieves and verifies the archive, computes metrics, writes to the ledger the form, metrics, tags, and a storage-persistence attribute, and signals a sibling search/index system for each new datapoint. Therefore, feasibility here is not a generic “blockchain compatibility” notion, but the practical ability to implement (or safely approximate) the required pipeline under smart-contract constraints while preserving auditability and integrity.

In the blockchain literature, feasibility is shaped by system constraints that include (i) performance limits and throughput constraints, (ii) storage growth and distribution overhead, and (iii) challenges in connecting on-chain logic to external services and data (oracle-style issues). These constraints make heavy computation and large state growth poor candidates for on-chain implementation, often motivating hybrid architectures in which bulk data remains off-chain and the chain stores commitments and compact metadata. Smart-contract ecosystems add further constraints: execution is costed (gas), code and execution patterns are shaped by platform limits, and determinism is required. Empirical work on Solidity metrics shows that many metric distributions are truncated and exist in restricted ranges due to platform constraints,

which supports treating feasibility as a distinct and non-trivial selection criterion rather than assuming traditional software metrics transfer unchanged [1, 10, 8].

6.1.2 Definition: smart-contract feasibility

We define **smart-contract feasibility** of a candidate metric as:

The ability to compute, verify, and store a metric (or its verified output) within the defined evidence workflow with bounded on-chain computation and storage, minimal reliance on unverifiable external inputs, strong auditability (re-checking from archived evidence), and explicit binding to developer identity/provenance.

This definition is intentionally *architecture-aware*. A candidate can be feasible in one of two ways:

- **On-chain computable feasibility:** the metric can be computed directly in the contract with small, bounded logic.
- **Hybrid feasibility (recommended for PoC):** the metric is computed off-chain from a verified artifact (hash+link), and the contract stores only compact metric values plus hash commitments to evidence/inputs to enable auditability.

The rubric below evaluates feasibility in a way that allows both patterns, while explicitly penalizing designs that require unverifiable oracles, frequent high-throughput writes, or large on-chain state.

6.1.3 Rubric structure and scoring

Each metric receives a score on 10 dimensions, each scored in $\{0, 1, 2\}$ (0 = poor feasibility, 2 = strong feasibility). The rubric is used **only after** the multi-metric comparative analysis, to select the single PoC metric.

How to use the rubric (procedure).

1. For each candidate metric M_i , write a one-paragraph “implementation story”: what are inputs, what is computed, what is stored, how often updated, who attests.
2. Score each rubric dimension using the guidance below.
3. Compute a total feasibility score and also record *exclusion conditions* (dimensions where score = 0 implies unacceptable risk for the PoC).
4. Select the PoC metric as the best trade-off between (i) empirical performance under C1–C4 and (ii) feasibility score, with tie-breaking favoring determinism, auditability, and bounded cost.

6.1.4 Feasibility dimensions (detailed)

Dimension	0–2	Scoring rule (detailed)
D1. Determinism of computation	0–2	2: metric computation is deterministic given the archived artifact and explicit parameters (window definition, tool version). 1: deterministic only under additional assumptions (toolchain pinned; minor nondeterminism mitigated). 0: depends on nondeterministic factors or ambiguous extraction (e.g., floating-point randomness, unstable parsing).
D2. Bounded on-chain computation	0–2	2: on-chain logic is constant-time or small bounded checks (signature format check, hash equality, record writes). 1: moderate bounded computation per update (limited loops over small arrays). 0: requires heavy analysis on-chain (AST parsing, full-history scanning) or unbounded loops.
D3. Bounded on-chain storage growth	0–2	2: store only compact values + commitments (hashes) + minimal metadata. 1: store moderate aggregates (rolling sums, bounded windows). 0: requires storing large histories/artifacts or per-commit detailed logs on-chain.
D4. Update frequency feasibility	0–2	2: metric can be updated per release or per time window (low write rate). 1: requires medium-frequency updates (e.g., per PR/issue) but can be batched. 0: requires high-frequency updates (per commit/event stream) and becomes throughput-limited.
D5. External data/oracle dependence	0–2	2: inputs are self-contained in the archived artifact + signed form, verifiable by hash commitments. 1: uses off-chain data but can be posted as signed attestations with clear trust model. 0: requires unverifiable external services or subjective judgments (strong oracle problem).

Table 5: Feasibility dimensions with detailed scoring rules (D1–D5).

Dimension	0–2	Scoring rule (detailed)
D6. Auditability and re-checking	0–2	2: third parties can recompute metric from archive retrieved via link and verified by on-chain hash; measurement report is committed. 1: reproducible with effort (tooling dependencies; partial evidence). 0: cannot be reliably reproduced or verified from public evidence.
D7. Identity binding and provenance	0–2	2: metric evidence is bound to developer identity via signed submissions (and the same identity used consistently across submissions). 1: partial binding (mapping from commit authors; weaker identity guarantees). 0: attribution cannot be trusted or can be trivially spoofed.
D8. Integrity of artifact linkage	0–2	2: strong commitment: archive hash stored on-chain and verified against retrieved archive; versioning explicit. 1: link is stable but hash commitment incomplete or relies on weak assumptions. 0: artifact cannot be pinned; link rot or mutable artifact breaks verification.
D9. Metric-value representation (range + precision)	0–2	2: metric stored as small integers or fixed-point with clear bounds; low risk of overflow/precision issues. 1: requires careful normalization/scale but feasible (e.g., store numerator/denominator separately). 0: requires floating point or complex statistics not representable safely on-chain.
D10. Adversarial resilience in a public ledger context	0–2	2: metric is hard/expensive to inflate without leaving detectable traces, especially with hash-anchored evidence. 1: partially gameable but mitigations exist (normalization, batching, cross-check with other metrics in study). 0: trivially gameable with low-cost manipulations (e.g., verbosity inflation) and weak detectability.

Table 6: Feasibility dimensions with detailed scoring rules (D6–D10).

6.1.5 Evidence checklist per dimension (what you must document)

To keep feasibility rigorous and evidence-based, each dimension must be supported by explicit evidence:

- **D1 Determinism:** specify exact inputs (archive hash, link, version id), the window definition, and (for off-chain computation) pin the tool version and configuration; describe how the PoC ensures the same artifact yields the same output.
- **D2/D3 Cost bounds:** list the contract operations required per datapoint (comparisons, storage writes); argue boundedness.
- **D4 Update frequency:** define update cadence (per release/time window). Explain why per-commit updates are avoided.
- **D5 Oracle dependence:** list any off-chain sources beyond the archive; if any exist, state whether they are hash-anchored or signed attestations.

- **D6 Auditability:** define the *re-check protocol*: retrieve archive, verify hash, recompute metric, compare with ledger value.
- **D7/D8 Provenance and linkage:** describe how the signed form + hash commitment prevents substitution of artifacts.
- **D9 Representation:** define encoding (int, fixed-point, store numerator/denominator); describe overflow handling and normalization.
- **D10 Adversarial model:** list low-cost manipulations and whether the system can detect them via evidence consistency.

6.1.6 Feasibility Exclusion Conditions

Even if a metric is strong empirically, it is a weak PoC candidate if it triggers exclusion conditions:

- **Exclusion Condition A:** score 0 on D1 (non-deterministic), D6 (non-auditable), or D8 (artifact cannot be pinned).
- **Exclusion Condition B:** requires high-frequency updates (D4=0) that exceed realistic ledger throughput.
- **Exclusion Condition C:** depends on unverifiable external facts (D5=0) without a signed-attestation model.

6.1.7 How this rubric maps to the project evidence workflow

The project workflow specifies a procedure where a smart contract: (i) checks the signature in the form, (ii) retrieves the archive from the link, (iii) checks signature and hash match, (iv) measures quality metrics, (v) writes form + metrics + tags + storage-persistence reference, and (vi) signals a sibling indexing system. The rubric operationalizes this in dimensions D1, D6, D7, and D8 (determinism, auditability, identity binding, artifact linkage), and constrains practical implementation via D2–D5 and D9 (bounded compute/storage, update frequency, oracle dependence, representation).

6.1.8 How the rubric is applied to the five thesis candidates (template only)

Table 7 is used after Phase II to score each candidate. Scores are not assigned in the current reporting phase.

Metric	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
M1 Snapshot size (LOC/SLOC)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M2 Patch complexity (churn)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M3 Activity cadence	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M4 Ownership/concentration	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M5 Maintenance responsiveness (design ext.)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Table 7: Feasibility rubric status table. All scores are deferred to Phase II comparative evaluation.

7 Future DESTINO PoC Integration Blueprint

This section records the intended integration shape for the future proof-of-concept without choosing the PoC metric. Its purpose is to keep Phase I connected to the deployment target while preserving the decision rule: empirical comparison and feasibility scoring must happen before implementation.

7.1 Design Objective and Boundary

The future PoC should support evidence-based developer reputation records that are traceable to hash-pinned software artifacts. The contract layer should not compute repository metrics directly. Repository traversal, Git history analysis, duration normalization, bot handling, and ownership calculations remain off-chain because they require file-system access, Git tooling, and potentially unbounded iteration. The contract stores the result and the commitments needed to audit it.

The boundary is:

- **Off-chain:** artifact retrieval, hash verification, metric extraction, cleaning, normalization, and report generation;
- **On-chain:** submission registration, compact metric-record storage, evidence-hash storage, permission checks, and events;
- **Not in Phase I:** final metric selection, contract implementation, and M5 maintenance-responsiveness implementation.

7.2 Metric-Agnostic Record Shape

The future contract interface can remain metric-agnostic until Phase II selects a metric. A minimal record shape is shown in Table 8; field names are implementation placeholders, not final Solidity declarations.

Record	Core fields	Role
ProjectSubmission	projectId, developerId, versionId, archiveLink, archiveHash, tags, storagePersistence, submittedAt	Registers the artifact/provenance claim that later metric evidence depends on.
MetricRecord	projectId, versionId, metricName, metricValue, evidenceHash, analysisPolicy, recordedAt	Stores the compact selected metric result and binds it to an auditable off-chain report.

Table 8: Metric-agnostic record shape for the future DESTINO PoC.

The analysisPolicy field is included because the current report now makes population and sensitivity rules explicit: human-only M3/M4 outputs, short-window flags, and bot-sensitivity summaries are not incidental cleaning details. They are part of the evidence interpretation that must accompany any future metric record.

7.3 Transaction and Replay Flow

The intended end-to-end flow is:

1. a developer registers signed project/archive metadata and an archive hash;
2. an off-chain verifier retrieves the artifact and checks hash equality;
3. the selected Phase II metric is computed using the pinned script version and declared analysis policy;
4. a compact metric value and evidence-report hash are recorded on-chain;
5. an event is emitted so an indexing/search component can expose the new datapoint;
6. auditors can replay the same computation from the artifact hash, script version, window definition, and evidence report.

This design preserves the core DESTINO requirement: trust is not based on a private metric calculation alone, but on a replayable chain of artifact commitments, deterministic scripts, and compact public records.

7.4 How Phase I Outputs Support the Blueprint

The current Phase I outputs are already shaped for this future integration:

- frozen projects, versions, and windows provide stable input identifiers;
- clean and human-only metric tables provide candidate values that can be reduced to compact records;
- duration flags and bot summaries document analysis policies that affect interpretation;
- logs and reproducibility checkpoints provide the replay trail for third-party verification;
- the feasibility rubric defines the gate that prevents an empirically attractive but contract-hostile metric from becoming the PoC.

This means the current report completes the DESTINO alignment at the planning and evidence level. The implementation step remains properly deferred until Phase II identifies the metric with the best empirical and feasibility tradeoff.

8 Phase I: Dataset Freeze and Version-Window Definition

This section documents the completed Phase I freeze workflow for the Initial Cohort: repository freeze, stable-tag freeze, and consecutive-window construction.

8.1 Repository Freeze

The Initial Cohort includes four mature Python repositories hosted on GitHub. Storage persistence is recorded as High (GitHub) for all projects. Software-type tags are minimal and manually assigned.

Cohort	Project ID	Repository URL	Storage persistence	Software-type tags	Notes
Initial	flask	https://github.com/pallets/flask	High (GitHub)	web-framework, library	Stable release history with dense tags.
Initial	requests	https://github.com/psf/requests	High (GitHub)	http-client, library	Repository-specific clone behavior documented in extraction notes.
Initial	urllib3	https://github.com/urllib3/urllib3	High (GitHub)	http-client, library	Strong release cadence, mature maintainer base.
Initial	click	https://github.com/pallets/click	High (GitHub)	cli-toolkit, library	Highest churn within the Initial Cohort.

Table 9: Frozen Initial Cohort dataset (`dataset_freeze_round1.csv`).

8.2 Tag Freeze and Consecutive Windows

For each project, we freeze eight stable release tags. Let $\{T_1, \dots, T_8\}$ be tags sorted chronologically (oldest to newest). We define seven windows $W_k = (T_k \rightarrow T_{k+1})$ for $k \in \{1, \dots, 7\}$. Each tag is bound to an immutable commit hash. The repository freeze is captured in `dataset_freeze_round1.csv`, and the tag/window freeze is captured in `versions.csv` and `windows.csv`, ensuring exact reproducibility.

Initial Cohort totals are:

- 32 frozen tags (4 projects $\times$ 8 tags);
- 28 consecutive windows (4 projects $\times$ 7 windows);

- tag-date coverage from 2023-05-03 to 2026-02-19.

Project	Frozen tags	Windows	Tag-date coverage
click	8	7	2023-07-18 to 2025-11-15
flask	8	7	2023-09-30 to 2026-02-19
requests	8	7	2023-05-03 to 2025-08-18
urllib3	8	7	2024-09-12 to 2026-01-07

Table 10: Initial Cohort version coverage extracted from `versions.csv` and `windows.csv`.

Project	Min window days	Max window days	Mean window days
click	10.04	489.99	121.58
flask	16.05	219.96	124.66
requests	0.26	376.04	119.74
urllib3	3.00	170.05	68.89

Table 11: Window-duration statistics from `windows.csv`. The high spread motivates the generated rate-normalized analysis outputs.

9 Phase I: Metric Extraction Pipeline

All candidate metrics M1–M4 are computed off-chain from Git history and tag windows. The extraction scripts are implemented as reproducible command-line programs that generate deterministic CSV outputs. The pipeline explicitly consumes the freeze artifacts `dataset_freeze_round1.csv`, `versions.csv`, and `windows.csv`.

9.1 Primary CSV outputs

- `loc_per_tag.csv`: per (project, tag) snapshot size (total, blank, comment, code lines).
- `metrics_window_dev.csv`: per (project, window, developer) commits, added/deleted, churn, activity days, first/last committer timestamps.
- `metrics_window_totals.csv`: per (project, window) totals and unique authors.
- `metrics_window_dev_clean.csv` and `metrics_window_totals_clean.csv`: cleaned canonical datasets used for analysis.
- `ownership_dev_shares_clean.csv` and `ownership_summary_clean.csv`: ownership and concentration computed from the clean dataset.
- Normalized metric outputs: `metrics_window_dev_normalized.csv` and `metrics_window_totals_normalized.csv`, with per-day and per-KLOC-day derived fields.
- Window-duration outputs: `window_duration_flags.csv` and `window_duration_sensitivity_summary.csv`, with short-window flags and sensitivity thresholds.
- Human-only and bot-sensitivity outputs: `metrics_window_dev_human.csv`, `metrics_window_totals_human.csv`, `ownership_summary_human.csv`, and `bot_summary_by_project.csv`.

9.2 Script-to-output mapping

Script	Primary output(s)
scripts/extract_window_metrics.py	metrics_window_dev.csv, metrics_window_totals.csv
scripts/compute_loc_per_tag.py	loc_per_tag.csv
scripts/clean_round1_metrics.py	metrics_window_dev_clean.csv, metrics_window_totals_clean.csv
scripts/recompute_ownership_from_clean.py	ownership_dev_shares_clean.csv, ownership_summary_clean.csv
scripts/compute_duration_normalized_metrics.py	metrics_window_dev_normalized.csv, metrics_window_totals_normalized.csv, window_duration_flags.csv
scripts/compute_bot_sensitivity_outputs.py	metrics_window_totals_human.csv, ownership_summary_human.csv, bot_summary_by_project.csv

Table 12: Executable mapping from pipeline scripts to canonical Initial Cohort outputs.

9.3 Metric definitions (implementation-level)

M1 (LOC/SLOC). For each frozen tag commit, we count Python *.py lines and classify lines into blank, comment, and code. Common non-source directories are excluded to preserve comparability.

M2 (Churn). For each window ($from_tag \rightarrow to_tag$), we use Git numstat to sum lines added and deleted per developer, and define $churn = added + deleted$. Binary file changes are tracked via `binary_file_count`.

M3 (Cadence). Cadence is represented by commit counts per developer-window and activity-days per developer-window. We store first/last committer timestamps for optional temporal analyses. Human-only cadence outputs are the primary developer-behavior population; all-account cadence outputs are retained for audit and sensitivity analysis.

M4 (Ownership and concentration). We compute developer shares within each project-window:

$$share_{d,w}^{(churn)} = \frac{churn_{d,w}}{\sum_{d'} churn_{d',w}}, \quad share_{d,w}^{(commits)} = \frac{commits_{d,w}}{\sum_{d'} commits_{d',w}}.$$

We derive top-1/top-3 shares and Gini concentration to characterize maintainership concentration. Because M4 represents human maintainership concentration, the primary M4 tables exclude rows flagged as automated accounts. All-account ownership outputs remain available to quantify the effect of this exclusion.

Duration-normalized derived metrics. Because release windows have uneven durations, the pipeline derives `commits_per_day`, `churn_per_day`, and `churn_per_kloc_day`. The normalization script also marks windows shorter than 1, 7, and 14 days. Windows shorter than 7 days are retained in the dataset but marked as `sensitivity_only` for comparative analyses that are sensitive to release cadence.

Bot-sensitivity derived metrics. The bot-sensitivity script preserves the all-account clean tables and exports a parallel human-only population. This resolves the analysis population explicitly: M3 and M4 use human-only outputs as the primary evidence, while all-account outputs and bot summaries document automation effects.

9.4 Error handling and traceability

The extraction pipeline is fault-tolerant at repository-window granularity:

- Git command failures are appended to `logs/errors.log` and processing continues;
- cleaning, ownership, duration-normalization, and bot-sensitivity scripts write structured run logs with row counts and validation status;
- all derived tables are built from declared input files only (no manual edits).

This logging policy provides an audit trail suitable for re-execution and for thesis appendix verification.

10 Phase I: Data Integrity Verification and Cleaning

10.1 Sanity checks

We verify:

- Window integrity: each project has seven consecutive windows from eight frozen tags.
- Arithmetic: for every row, $\text{churn} = \text{added} + \text{deleted}$.
- Totals: window totals equal sums of developer rows.

All 28 windows pass integrity checks, and no window has zero commits or zero churn in the cleaned totals.

10.2 Canonical identity

The regenerated extraction pipeline canonicalizes developer identity during extraction. The canonical aggregation key is:

`(project_id, window_id, author_email)`

The extractor aggregates commits by email and selects a canonical `author_name` using the most frequent non-unknown value. This produces `metrics_window_dev.csv` with 273 developer-window rows. The cleaning pass keeps the same 273 canonical rows and records 0 duplicate key groups merged.

10.3 Bot labeling

We retain an `is_bot` flag to allow reporting with and without automated accounts. In the Initial Cohort, 18 developer-window rows are flagged as bots in the cleaned dataset. The pipeline now exports both all-account and human-only analysis populations: all-account outputs remain the audit trail for total repository activity, while human-only outputs are the primary source for M3 cadence and M4 ownership/concentration. This prevents automated dependency-update or formatting accounts from being interpreted as human maintainers.

10.4 Known limitation: commit-date/tag-date drift

Some commits included between tags carry committer timestamps that precede the tag-date window boundaries. This can happen when commits were authored or committed on side branches before they became reachable in the tagged history. This affects optional cadence features based on timestamps (first/last commit), but does not affect churn or commit counts computed from the Git revision range. Phase II therefore treats human-only commit counts and duration-normalized rates as primary cadence evidence, and uses timestamp-derived active-day features only with sensitivity checks.

10.5 Issue-resolution ledger

Issue	Observed count	Initial Cohort resolution
Display-name variation for the same email/window	0 groups after regeneration	Canonical aggregation by email key is performed during extraction.
Incorrect unique-author counting in raw totals	0 windows after regeneration	Raw and clean totals compute unique authors by <code>author_email</code> .
Bot contamination in ownership shares	18 rows	Preserved <code>is_bot</code> , exported human-only M3/M4 outputs, and generated bot-summary CSVs for sensitivity analysis.
Commit timestamp outside tag-date boundaries	45 rows	Documented limitation; Phase II sensitivity checks avoid over-reliance on timestamp-derived cadence fields.

Table 13: Initial Cohort issue ledger and implemented cleaning/validation actions.

11 Initial Cohort Extraction Summary (Phase I Deliverables)

Initial Cohort extraction spans 28 windows and produces 273 clean developer-window rows after identity canonicalization. Aggregate all-account totals across all projects are 902 commits and 58,298 lines of churn. The primary human-only population for M3/M4 contains 255 developer-window rows, 759 commits, and 57,359 lines of churn.

Project	Windows	Total commits	Total churn	Mean authors/window	Bot rows
click	7	422	29,176	15.43	8
flask	7	180	13,159	6.71	2
requests	7	152	3,277	8.86	3
urllib3	7	148	12,686	8.00	5

Table 14: Initial Cohort summary from cleaned datasets (commits/churn totals per project; bot rows flagged separately).

Scope	Windows	Dev rows	Total commits	Total churn
All accounts	28	273	902	58,298
Human-only M3/M4 population	28	255	759	57,359

Table 15: Initial Cohort global extraction totals from all-account and human-only clean outputs.

Project	Min code lines	Max code lines	Mean code lines
click	13,454	16,504	14,691
flask	12,586	12,977	12,816
requests	7,964	8,425	8,257
urllib3	23,694	25,260	24,748

Table 16: Snapshot size statistics (code lines) across the 8 frozen tags per project (M1).

12 Detailed Descriptive Results for M1–M4 (Initial Cohort)

This section reports detailed descriptive outputs already computed from the clean Phase I Initial Cohort datasets. These are not comparative rankings and do not select a PoC metric; they document empirical behavior that will be used in Phase II.

12.1 M1 Snapshot size evolution (LOC/SLOC)

Project	First code	Last code	Delta (%)	Min	Max	Mean
click	13454	16504	+3050 (22.67%)	13454	16504	14691.38
flask	12586	12977	+391 (3.11%)	12586	12977	12815.75
requests	7964	8412	+448 (5.63%)	7964	8425	8256.75
urllib3	23694	25260	+1566 (6.61%)	23694	25260	24747.62

Table 17: M1 evolution across frozen tags (values from `out/loc_per_tag.csv`; first/last are chronological tag endpoints).

M1 indicates moderate-to-strong size growth in all projects during the frozen interval, with click showing the largest relative increase.

12.2 M2 Churn and M3 cadence intensity by project

Project	Commits	Churn	Churn/commit	Window days	Commits/day	Churn/day	Mean authors/window
click	367	28901	78.75	851.03	0.431	33.96	14.29
flask	156	12935	82.92	872.60	0.179	14.82	6.43
requests	125	3115	24.92	838.20	0.149	3.72	8.43
urllib3	111	12408	111.78	482.24	0.230	25.73	7.29

Table 18: M2/M3 human-only project-level intensity derived from `out/metrics_window_totals_human.csv` and `data/windows.csv`.

These values show that Initial Cohort projects differ substantially in both human change magnitude (M2) and human activity pace (M3). All-account totals remain useful as descriptive repository-activity evidence, but M3 developer-behavior analysis uses the human-only population. Duration-normalized outputs are therefore generated as explicit Phase I artifacts rather than deferred to later analysis.

12.3 Bot-sensitivity outputs

Project	Bot rows	Bot commit %	Bot churn %	Windows with bots	Bot top-commit windows
click	8	13.03	0.94	6	1
flask	2	13.33	1.70	1	0
requests	3	17.76	4.94	3	2
urllib3	5	25.00	2.19	5	1

Table 19: Bot impact summary from `out/bot_summary_by_project.csv`. Percentages are relative to all-account project totals.

Across the Initial Cohort, bot rows contribute 143 of 902 commits (15.85%) but only 939 of 58,298 churn lines (1.61%). This means bot handling matters most for commit-frequency and ownership/cadence interpretations, not for aggregate churn magnitude. The primary M3 and M4 evidence therefore uses human-only outputs, while all-account outputs remain available for audit and sensitivity checks.

12.4 Duration-normalized analysis outputs

Sensitivity threshold	Window count	Affected windows
<1 day	2	requests_W03; requests_W04
<7 days	4	requests_W03; requests_W04; urllib3_W05; urllib3_W06
<14 days	6	click_W04; requests_W03; requests_W04; requests_W05; urllib3_W05; urllib3_W06

Table 20: Short-window sensitivity flags. Windows below seven days are retained but marked as `sensitivity_only`.

Window	Duration days	Commits/day	Churn/day	Analysis set
click_W04	10.04	1.793	290.15	primary
click_W06	45.05	1.709	104.22	primary
click_W03	142.22	1.069	84.46	primary
requests_W04	0.86	3.481	63.83	sensitivity_only
click_W07	63.62	0.864	49.18	primary
requests_W03	0.26	3.784	45.41	sensitivity_only

Table 21: Highest churn/day windows from `out/metrics_window_totals_normalized.csv`. Very short windows can inflate rates and are explicitly flagged for sensitivity analysis.

The generated normalized tables provide aggregate per-day commit and churn rates, including a per-KLOC-day churn rate, and parallel per-day fields for developer-window analysis. Raw totals remain useful as descriptive evidence, while duration-sensitive comparisons use the normalized artifacts.

12.5 M3 Developer activity-day profile

Project	Mean active_days/dev-window	Min	Max	Min window days	Max window days
click	2.10	1	19	10.04	489.99
flask	1.84	1	17	16.05	219.96
requests	1.54	1	8	0.26	376.04
urllib3	1.75	1	13	3.00	170.05

Table 22: M3 human-only cadence-related activity profile from `out/metrics_window_dev_human.csv` and `data/windows.csv`.

The strong window-duration heterogeneity reinforces that cadence comparisons should use the generated per-day fields and timestamp-boundary sensitivity checks.

12.6 M4 Ownership and concentration profile

Project	Mean top1 commits	Mean top3 commits	Mean top1 churn	Mean top3 churn
click	0.363	0.689	0.502	0.862
flask	0.708	0.885	0.891	0.994
requests	0.573	0.791	0.713	0.936
urllib3	0.629	0.783	0.695	0.937

Table 23: M4 human-only share concentration means from out/ownership_summary_human.csv.

Project	Mean Gini commits	Mean Gini churn	Max top1 churn
click	0.506	0.716	0.702
flask	0.493	0.680	0.991
requests	0.217	0.408	1.000
urllib3	0.310	0.496	1.000

Table 24: M4 human-only inequality indicators from out/ownership_summary_human.csv.

M4 highlights non-trivial human concentration across windows, with some windows reaching full single-developer top-1 churn share in requests and urllib3.

13 Phase I Execution Ledger (Detailed, Step-Mapped)

This section documents the exact operational sequence executed in Phase I for the Initial Cohort, aligned with steps 2.1–2.5. It provides a direct bridge between roadmap instructions and reproducible artifacts.

Step	Action	Primary artifact(s)	Verification result
2.1	Freeze repository dataset metadata	data/dataset_freeze_round1.csv	4 projects frozen with software-type tags and storage-persistence metadata.
2.2	Freeze stable tags (8 per project)	data/versions.csv	32 rows with tag name, commit hash, and tag date.
2.3	Build consecutive windows (7 per project)	data/windows.csv	28 windows generated in chronological order from frozen tags.
2.4.2	Extract raw developer-window metrics (M2/M3 base)	out/metrics_window_dev.csv; out/metrics_window_totals.csv	273 canonical developer rows; 28 window totals; logs/errors.log remained empty.
2.4.3	Compute LOC/SLOC snapshots (M1)	out/loc_per_tag.csv	32 per-tag rows produced from frozen commits.
2.4.4	Compute ownership/concentration (M4) from clean base	out/ownership_dev_shares_clean.csv; out/ownership_summary_clean.csv	273 developer-share rows and 28 summary rows with top shares and Gini.
2.4.5	Compute duration-normalized analysis fields	out/metrics_window_totals_normalized.csv; out/metrics_window_dev_normalized.csv; out/window_duration_flags.csv	28 normalized window rows, 273 normalized developer rows, and short-window sensitivity flags generated.
2.4.6	Compute bot-sensitivity and human-only outputs	out/metrics_window_dev_human.csv; out/ownership_summary_human.csv; out/bot_summary_by_project.csv	255 human developer rows, 28 human ownership summaries, and bot impact summaries generated.
2.5	Cleaning and sanity checks	out/metrics_window_dev_clean.csv; out/metrics_window_totals_clean.csv; logs/*.log	Duplicate identity groups merged=0 after extraction-time canonicalization; clean rows=273; all validation checks passed.

Table 25: Detailed execution ledger for Phase I Initial Cohort pipeline operations.

13.1 Cleaning and validation outcomes

The cleaning and ownership recomputation logs provide concrete numerical guarantees:

- input developer-window rows: 273; cleaned rows: 273;
- duplicate key groups merged during cleaning: 0 (email-based canonicalization is performed during extraction);
- windows with zero churn: 0; windows with zero commits: 0;
- bad windows for share-sum checks: 0;
- bad windows for unique-author consistency: 0;
- duration-normalized rows generated: 28 aggregate-window rows and 273 developer-window rows;
- very short windows flagged for sensitivity analysis: 4 windows below seven days;
- bot-flagged rows retained in clean data: 18 (click 8, flask 2, requests 3, urllib3 5);
- human-only rows generated for primary M3/M4 analysis: 255;
- bot contribution: 143 commits (15.85%) and 939 churn lines (1.61%).

These outcomes confirm that Phase I outputs are analysis-ready and suitable as auditable input for Phase II.

14 Reproducibility Protocol and Data Lineage

14.1 Artifact layout

Initial Cohort artifacts are organized under a single workspace root (e.g., `phase1_round1/`, equivalent to the `month2_round1/` folder in execution notes) with fixed subdirectories: `data/`, `repos/`, `scripts/`, `out/`, and `logs/`. This layout is intentionally stable so that all extraction and cleaning steps can be rerun without changing paths or script arguments.

14.2 Execution protocol

The pipeline is deterministic given frozen `versions.csv` and `windows.csv`. The execution order is:

1. `python scripts/extract_window_metrics.py`
2. `python scripts/compute_loc_per_tag.py`
3. `python scripts/clean_round1_metrics.py -windows data/windows.csv`
4. `python scripts/recompute_ownership_from_clean.py`
`-in out/metrics_window_dev_clean.csv -outdir out -windows data/windows.csv`
5. `python scripts/compute_duration_normalized_metrics.py`
6. `python scripts/compute_bot_sensitivity_outputs.py`

14.3 Lineage checkpoints

Artifact	Rows	Role
<code>data/versions.csv</code>	32	Frozen tags and commit hashes (4 projects $\times$ 8 tags).
<code>data/windows.csv</code>	28	Consecutive windows (4 projects $\times$ 7 windows).
<code>out/metrics_window_dev.csv</code>	273	Canonical developer-window extraction from Git history.
<code>out/metrics_window_dev_clean.csv</code>	273	Validated developer-window dataset after cleaning checks.
<code>out/metrics_window_totals_clean.csv</code>	28	Window totals from clean developer rows.
<code>out/loc_per_tag.csv</code>	32	M1 snapshot-size values per frozen tag.
<code>out/ownership_summary_clean.csv</code>	28	All-account concentration statistics (top shares and Gini) from clean data.
<code>out/metrics_window_totals_normalized.csv</code>	28	Duration-normalized aggregate metrics and per-KLOC-day derived fields.
<code>out/metrics_window_dev_normalized.csv</code>	273	Duration-normalized developer-window metrics.
<code>out/window_duration_flags.csv</code>	28	Short-window flags and primary/sensitivity analysis labels.
<code>out/metrics_window_dev_human.csv</code>	255	Human-only developer-window dataset for M3/M4 primary analysis.
<code>out/metrics_window_totals_human.csv</code>	28	Human-only window totals derived from clean developer rows.
<code>out/ownership_summary_human.csv</code>	28	Human-only ownership/concentration statistics for M4.
<code>out/bot_summary_by_project.csv</code>	4	Project-level automation contribution and sensitivity summary.
<code>out/bot_summary_by_window.csv</code>	28	Window-level automation contribution and top-contributor flags.

Table 26: Initial Cohort data lineage checkpoints used for reproducibility and auditing.

14.4 Automated validation evidence

Validation evidence is recorded in log files and confirms that:

- duplicate-key groups merged in cleaning: 0;
- windows with zero total churn: 0;
- windows with zero total commits: 0;
- windows failing share-sum validations (churn or commits): 0;
- windows failing unique-author consistency checks: 0;
- windows shorter than seven days and flagged as `sensitivity_only`: 4;
- all-account total mismatches found during bot-sensitivity validation: 0;
- bot rows retained for audit: 18; human-only rows exported for primary M3/M4 analysis: 255.

These checks establish that downstream Phase II comparative analysis will operate on a coherent and fully auditable dataset.

15 Threats to Validity

15.1 Construct validity

M2 uses churn as a proxy for change magnitude, which does not directly encode semantic complexity or defect risk. M3 cadence features are sensitive to timestamp semantics; commit-date/tag-date drift is documented and treated as a robustness concern. Automated accounts can also inflate commit-frequency and ownership/cadence interpretations, so M3 and M4 use human-only outputs as the primary population and keep all-account outputs for sensitivity checks. To reduce construct ambiguity, the study keeps raw fields (added, deleted, commits, activity days) alongside derived indicators.

15.2 Internal validity

Potential extraction errors were mitigated by deterministic windows, explicit cleaning rules, and arithmetic checks ($\text{churn} = \text{added} + \text{deleted}$). Developer identity is canonicalized by email during extraction, window totals are recomputed and validated from clean developer rows, and bot-sensitivity outputs validate that all-account totals remain consistent before deriving human-only tables.

15.3 External validity

The Initial Cohort includes four mature Python projects only, so generalization to other ecosystems is limited. This is intentional: Phase I focuses on pipeline correctness; the Extended Cohort adds six more repositories to test transferability across a larger and more heterogeneous sample.

15.4 Conclusion validity

No inferential claims are made in Phase I. Comparative conclusions are deferred to Phase II and will be supported by explicit stability/ranking analyses and the predefined feasibility rubric. This sequencing reduces risk of premature metric selection bias.

16 Phase I Conclusion

Phase I for the Initial Cohort is complete at the dataset and pipeline level. The thesis now has:

- frozen repositories, tags, and windows (fully reproducible inputs);
- extracted and cleaned developer-window metrics for M1–M4;
- ownership/concentration outputs computed strictly from canonical clean data;
- duration-normalized analysis outputs and short-window sensitivity flags for uneven release windows;
- human-only M3/M4 outputs and bot-sensitivity summaries, so automation is handled explicitly;
- a DESTINO-aligned evidence boundary and future PoC integration blueprint;
- validation logs showing arithmetic, identity, and aggregation consistency.

At this stage, the work is intentionally descriptive and methodological. No PoC metric is selected yet. Selection is explicitly deferred to Phase II after comparative evaluation under the criteria in Section 4 and feasibility scoring in Section 6.1.

17 Planned Next Phase: Comparative Metric Evaluation

Phase II will perform comparative analysis across all candidate metrics using the criteria defined in Section 4, including stability and ranking consistency across consecutive windows. Only after this comparative evaluation will we apply the smart-contract feasibility rubric and select a single metric for the baseline PoC implementation.

17.1 Phase II execution plan

1. Compute criterion evidence (C1–C4) for all candidates using the clean, human-only, bot-sensitivity, and duration-normalized Initial Cohort outputs.
2. Report full-dataset and robust results using the generated short-window sensitivity flags where needed.
3. Add the Extended Cohort repositories and rerun the full extraction/cleaning pipeline for external-validity support.
4. Score feasibility dimensions for each candidate using the predefined rubric.
5. Select exactly one PoC metric using the decision protocol in Section 4.7.
6. Instantiate the selected metric inside the DESTINO-aligned metric-record blueprint in Section 7.

17.2 Decision gate criteria

The PoC metric will be selected only if it simultaneously satisfies:

- no feasibility exclusion criterion triggered (Section 6.1);
- acceptable comparative behavior under C1–C4 relative to alternatives;
- reproducible extraction with no unresolved data-integrity issues.

This gate prevents premature implementation based on a single attractive but methodologically weak indicator.

References

- [1] Weichao Gao, William G. Hatcher, and Wei Yu. A survey of blockchain: Techniques, applications, and challenges. In *Proceedings of the 27th International Conference on Computer Communication and Networks (ICCCN)*, 2018.
- [2] Gerta Kapllani, Ilya Khomyakov, Ruzilya Mirgalimova, and Alberto Sillitti. An empirical analysis of the maintainability evolution of open source systems. In *Open Source Systems*. Springer, 2020.
- [3] Eleni Koutrouli and Aphrodite Tsalgatidou. Reputation systems evaluation survey. *ACM Computing Surveys*, 48(3):35:1–35:28, 2015.
- [4] Balraj Kumar. A survey of key factors affecting software maintainability. In *2012 International Conference on Computing Sciences*, 2012.
- [5] Xiaozhou Li, Sergio Moreschini, Zheyang Zhang, and Davide Taibi. Exploring factors and metrics to select open source software components for integration: An empirical study. *Journal of Systems and Software*, 188:111255, 2022.
- [6] Andreas Schlosser, Marco Voss, and Lars Brueckner. Comparing and evaluating metrics for reputation systems by simulation. Conference manuscript, 2004. Local PDF copy in the “Papers to read” collection.
- [7] Sharifah Mashita Syed-Mohamad, Amir Ngah, Al-Fahim Mubarak Ali, and Pantea Keikhosrokiani. Measuring software maintainability: An analysis of evolving metrics across development paradigms. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 63(2):181–195, 2026.
- [8] Masoud Jamshidiyan Tehrani. Assessing vulnerability in smart contracts: The role of code complexity metrics in security analysis. arXiv preprint, 2026. arXiv:2411.17343v4.
- [9] Aline Lopes Timóteo, Alexandre Álvaro, Eduardo Santana de Almeida, and Silvio Romero de Lemos Meira. Software metrics: A survey. Survey manuscript. Local PDF copy in the “Papers to read” collection.
- [10] Roberto Tonelli, Giuseppe Antonio Pierro, Marco Ortu, and Giuseppe Destefanis. Smart contracts software metrics: A first study. *PLOS ONE*, 18(2):e0281043, 2023.